API Stream de Java 8 (/API-Stream-Java8.html)

April 17, 2017 streams collections

Codigo fuente disponible (https://github.com/windoctor7/codigo-tutoriales-blog/tree/master/java-streams)

Más que un cookbook esta entrada se trata de un **workshop**, por lo tanto deberás bajar el código fuente base y por tu cuenta ir escribiendo el código que encontrarás en este post.

1. Introducción

Antes de iniciar es indispensable bajar el código fuente base de este ejercicio. El proyecto está construido con gradle por lo tanto lo podrás importar con eclipse, netbeans o IntelliJ IDEA (https://www.jetbrains.com/idea/).

Dentro del proyecto encontrarás un archivo csv que usaremos para los ejercicios. Una vez que tengas importado el proyecto en tu IDE preferido, ejecuta la clase Main para asegurarte que todo está en órden. En la consola verás los nombres de los productos que se cargaron desde un archivo csv ubicado en la carpeta resources.

Será conveniente **pero no indispensable** que conozcas un poco de las expresiones lamda (https://www.adictosaltrabajo.com/tutoriales/expresiones-lambda-con-java-8/) que también son una característica nueva en Java 8.

Si lo deseas puedes cargar el archivo csv a una base de datos relacional como SQLite e ir ejecutando las consultas que encontrarás y comparar los resultados con los que obtendrás usando el API Stream de Java.

La estructura del archivo csv es la siguiente,

productID	productName	supplierID	categoryID	unitPrice	unitsInStock
1	Chai	1	1	18	39
2	Chang	1	1	19	17
3	Aniseed Syrup	1	2	10	13
4	Chef Anton's Cajun Seasoning	2	2	22	53
5	Chef Anton's Gumbo Mix	2	2	21.35	0

Si ya tienes el proyecto importado en tu IDE favorito, puedes comenzar el workshop.

---

2. Interfaz Stream

La clave de todo es la interfaz Stream (https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html). En español la palabra Stream quiere decir **flujo** o **secuencia** y según la documentación oficial de Java, la interfaz Stream representa una secuencia de elementos, pero ¿qué clase de elementos?, elementos del tipo que indiquemos. La interfaz Stream es genérica, por lo tanto para recuperar un stream de nuestros productos podemos hacer esto,

```
Stream<Product> productStream = products.stream(); //flujo de elementos Product
productStream.forEach(product -> System.out.println(product)); // imprime la lista de pr
oductos
// productStream.forEach(System.out::println); esta linea es equivalente a la anterior
```

En este workshop realizaremos algunas operaciones que aplicamos comúnmente a una base de datos relacional usando sql, pero las haremos sobre listas en Java usando Stream, por lo tanto a continuación te presento una tabla donde se muestran algunos métodos de la interfaz `Stream` junto a una posible equivalencia en sql. No te detengas mucho en revisarla, mejor regresa a ella conforme vayas teniendo dudas a lo largo de este workshop.

SQL	**Interfaz Stream**
from	stream()
select	map()
where	filter() (antes de un collecting)
order by	sorted()
distinct	distinct()
having	filter() (después de un collecting)
join	flatMap()
union	concat().distinct()
offset	skip()
limit	limit()
group by	collect(groupingBy())
count	count()

3. Consultas simples

Considera la siguiente consulta **sql**,

```
    select name from products
```

El equivalente usando Java **Streams** es,

```
List<Product> products;
...
Stream<String> streams = products.stream().map(Product::getName);
```

- Con el método `stream()` obtenemos una secuencia de elementos de tipo `Product`. Este es el **from**.
- Con el método `map` recuperamos solo el atributo *name*. Este es el **select**.

El punto clave es obtener un Stream mediante el método `stream()` y a partir de ahí ejecutar las operaciones como filtrados, agrupaciones, etc.

---

4. Filtrado

> Recuperar los nombres de productos que tengan una existencia en el almacen menor a 10 unidades.

En **sql**,

```
   select name from products where units_in_stock < 10
```

Con Java **Streams**,

```
Stream<String> streams = products.stream().filter(p -> p.getUnitsInStock()<10).map(Produ
ct::getName);
streams.forEach(product -> System.out.println(product)); //imprime el resultado en conso
la
```

Es importante notar el órden en el que aparecen los métodos, primero se encuentra filter y después map. ¿Qué ocurre si colocamos primero a map y luego a filter?

```
//ERROR DE COMPILACION
Stream<String> streams = products.stream().map(Product::getName).filter(p -> p.getUnitsI
nStock()<10);
```

Obtendremos un error de compilación. ¿Por qué? Porque el método map devuelve el nombre del producto que es un String y la clase String no tiene un método que se llame filter.

El método `filter()` recibe un predicado. Un predicado es solo una función que devuelve un valor boolean y la instrucción `p.getUnitsInStock()<10` es una expresión booleana.

Si nuestra lógica es más compleja podemos considerar usar un método en lugar de lamdas.

```
...
Stream<String> streams = products.stream()
                            .filter(predicado()) //invocamos a un método de predicado
                            .map(Product::getName);
...

public Predicate<Product> predicado(){
    return new Predicate<Product>() {
        @Override
        public boolean test(Product product) {
            //aqui la logica requerida para devolver true o false
            return product.getUnitsInStock() < 10;
        }
    };
}
```

---

5. Ordenacion

> Obtener los nombres de productos que tengan una existencia menor a 10 unidades en el almacen pero ordenados de forma ascendente, es decir, de menor existencia a mayor existencia.

En **sql**

```
   select name from products where units_in_stock < 10
   order by units_in_stock asc
```

Con Java **Streams**

```
Stream<String> streams = products.stream()
                .filter(p -> p.getUnitsInStock()<10)
                .sorted(Comparator.comparingDouble(Product::getUnitsInStock))
                .map(Product::getName)
                ;
```

El método `sorted` recibe un `Comparator`. Ésta misma interfaz `Comparator` tiene algunos métodos que nos serán de gran ayuda

- `comparingInt()` Permite comparar elementos de tipo int
- `comparingDouble()` Permite comparar elementos de tipo double
- `comparingLong()` Permite comparar elementos de tipo long
- `thenComparing()` Permite anidar comparaciones. Útil cuándo deseamos ordenar por más de 1 atributo (ejemplo más adelante)

Lo mejor será revisar la documentación de la interfaz Comparator (https://docs.oracle.com/javase/8/docs/api/java/util/Comparator.html)

Si deseamos ordenar en forma descendente necesitamos aplicar un reverso,

```
Stream<String> streams = products.stream()
                .filter(p -> p.getUnitsInStock()<10)
                .sorted(Comparator.comparingDouble(Product::getUnitsInStock).reversed())
                .map(Product::getName)
                ;
```

Otra forma diferente de ordenar, es que nuestra clase Product implemente a la interfaz `Comparable`

```
public class Product implements Comparable<Product>{
    ...
    // atributos, setters/getters
    ...

    @Override
    public int compareTo(Product p) {
        if(this.getUnitsInStock() < p.getUnitsInStock())
            return -1;
        else if(this.getUnitsInStock() > p.getUnitsInStock())
            return 1;
        else
            return 0;
    }
}
```

Ahora al método `sorted()` es invocado sin argumentos,

```
Stream<String> streams = products.stream()
        .filter(p -> p.getUnitsInStock()<10)
        .sorted()
        .map(Product::getName)
        ;
```

¿Cómo ordenamos en forma descendente? Usando Comparator.reverseOrder()

```
Stream<String> streams = products.stream()
        .filter(p -> p.getUnitsInStock()<10)
        .sorted(Comparator.reverseOrder())
        .map(Product::getName)
        ;
```

¿Y si queremos ordenar por unitsInStock de forma descendente y por nombre de producto de forma ascendente?

En **sql**,

```
select productName, unitsInStock from products
where unitsInStock < 10
order by unitsInStock desc, productName asc;
```

Con Java **Streams**

```
Stream<String> streams = products.stream()
        .filter(p -> p.getUnitsInStock()<10)
        .sorted(
            Comparator //recordar que el método sorted recibe un Comparator.
                .comparing(Product::getUnitsInStock)
                .reversed() //invertimos el orden, será de mayor a menor
                .thenComparing(Product::getName) //una vez ordenado por unitsInStock, en
tonces ordenamos por nombre
        )
        .map(Product::getName)
        ;
```

Y si ahora queremos invertir las cosas y ordenar por unitsInStock de forma ascendente y por nombre de forma descendente?

En **sql**,

```
select productName, unitsInStock from products
where unitsInStock < 10
order by unitsInStock asc, productName desc;
```

Con Java Streams podríamos pensar en solo cambiar la posición del método `reversed()`

```
Stream<String> streams = products.stream()
        .filter(p -> p.getUnitsInStock()<10)
        .sorted(
            Comparator
                .comparing(Product::getUnitsInStock)
                .thenComparing(Product::getName)
                .reversed()

        )
        .map(Product::getName)
        ;
```

Sin embargo esto no es correcto ya que estamos ordenando de forma descendente por ámbos atributos, unitsInStock y name.

La forma correcta es aplicar el reverse sólo al campo **name**,

```
Stream<String> streams = products.stream()
        .filter(p -> p.getUnitsInStock()<10)
        .sorted(
                Comparator
                        .comparing(Product::getUnitsInStock) //ordenamos ascendente por
 unitsInStock
                        .thenComparing( // despues ordenamos por otro campo
                            Collections.reverseOrder( // pero este segundo campo sera po
r orden descendente
                                Comparator.comparing(Product::getName) // el segundo cam
po a ordenar
                            )
                        )
        )
        .map(Product::getName)
        ;
```

Hasta este punto podemos resumir lo siguiente:

- El método `sorted()` recibe un `Comparator`
- La interfaz `Comparator` nos proporciona algúnos métodos que nos serán útiles para las ordenaciones.
- Existe una clase `Collections` que tiene un método `reverseOrder()` el cual devuelve un `Comparator` que impone el reverso de una ordenación.
- Hay que tener cuidado donde se aplican las operaciones como reversos ya que podríamos aplicarlos a toda la colección y no a los campos que deseamos.

6. Agrupado

En SQL las operaciones como sum, max, min, avg, group by, partition by, etc., se llaman funciones de agregado. En Java, se especifican en el método `collect`

> Obtener el número de productos agrupados por proveedor.

En **sql**,

```
Select count(1), supplierID from products
GROUP BY  supplierID
```

Con Java **Streams**,

```
Map<Integer, Long> collect = products.stream()
        .collect( //en el metodo collect se especifican las funciones de agregacion
                Collectors.groupingBy( // deseamos agrupar
                        Product::getSupplier, // agrupamos por proveedor
                        Collectors.counting() // realizamos el conteo
                    )
                );

collect.forEach((s, c) -> System.out.printf("proveedor: %s: productos: %s \n", s,c));

// proveedor: 1: productos: 3
// proveedor: 2: productos: 4
// proveedor: 3: productos: 3
// proveedor: 4: productos: 3
// proveedor: 5: productos: 2
....
....
```

Dado que en el método `collect` especificamos funciones de agregado, casi siempre nos auxiliaremos de la clase `Collectors` la cuál nos proporciona varios métodos de funciones de agregado. En este ejemplo, usamos el método `groupingBy`

Si deseamos filtrar todos los productos que en almacen tengan menos de 20 unidades de existencia y agrupados por existencia,

```
Map<Integer, List<Product>> collect = products.stream()
        .filter(p -> p.getUnitsInStock() < 20)
        .collect(Collectors.groupingBy(Product::getUnitsInStock));

collect.forEach((unidades, producto) -> System.out.printf("existencias: %s Productos: %s
\n", unidades, producto));
;

// existencias: 0 Productos: [Product{id=5, name='Chef Anton's Gumbo Mix'.....},Product
{id=17, name='Alice Mutton',...}]
// existencias: 3 Productos: [Product{id=21, name='Sir Rodney's Scones'....}]
...
...
```

7. Sumas

> Obtener la suma del precio unitario de todos los productos agrupados por el número de existencias en el almacen.

En **sql**,

```
Select  unitsInStock, sum(unitPrice) from products
GROUP BY unitsInStock
```

Con Java **Streams**,

```
Map<Integer, Double> collect = products.stream()
        .collect( //en el metodo collect se especifican las funciones de agregacion
                Collectors.groupingBy( // deseamos agrupar
                        Product::getUnitsInStock, //agrupamos por existencias en stock
                        Collectors.summingDouble( //el tipo de dato a sumar es double
                                Product::getUnitPrice //sumamos el precio unitario
                        )
                )
        );

collect.forEach((stock, suma) -> System.out.printf("en stock: %s: suma: %s \n", stock,su
ma));

// en stock: 0: suma: 229.44
// en stock: 3: suma: 10.0
// en stock: 4: suma: 27.0
// en stock: 5: suma: 9.5
// en stock: 6: suma: 12.5
...
...
```

8. Having

Tomando el ejemplo anterior, le agregaremos algo más,

> Obtener la suma del precio unitario de todos los productos agrupados por el número de existencias en el almacen, pero solo obtener aquellos registros cuya suma sea mayor a 100.

En **sql**,

```
Select  unitsInStock, sum(unitPrice) from products
GROUP BY unitsInStock
HAVING sum(unitPrice) > 100
```

Con Java **Streams**,

```
List<Map.Entry<Integer, Double>> entryList = products.stream()
        .collect( //en el metodo collect se especifican las funciones de agregacion
                Collectors.groupingBy( // deseamos agrupar
                        Product::getUnitsInStock, //agrupamos por existencias en stock
                        Collectors.summingDouble( //sumamos el precio unitario el cual e
s tipo double
                                Product::getUnitPrice // agrupamos por proveedor
                        )
                )
        ).entrySet()
        .stream() //volvemos a generar un stream
        .filter(p -> p.getValue() > 100) //filtramos (simula el having)
        .collect(Collectors.toList());

entryList.forEach(list -> System.out.printf("en stock: %s, suma: %s\n",list.getKey(), li
st.getValue()));

// en stock: 0, suma: 229.44
// en stock: 17, suma: 377.8
// en stock: 26, suma: 117.1
// en stock: 29, suma: 114.45
```

9. Más operaciones

Promedio de existencias en almacen

```
Double average = products.stream()
                .collect(Collectors.averagingInt(Product::getUnitsInStock));
System.out.printf("Promedio de existencias en almacen: %s",average );

// Promedio de existencias en almacen: 40.54545454545455
```

Producto con el precio unitario más alto

```
Optional<Product> product = products.stream().max(Comparator.comparing(Product::getUnitP
rice));
System.out.println(product.get());

// Product{id=38, name='Côte de Blaye', supplier=18, category=1, unitPrice=263.5, unitsI
nStock=17}
```

Podemos obtener el **count, sum, min, max** y **average** con una sola operación. Por ejemplo si queremos obtener estas estadísticas respecto al precio unitario

```
DoubleSummaryStatistics statistics =
                products.stream().collect(Collectors.summarizingDouble(Product::getUnitP
rice));

System.out.println(statistics);

// DoubleSummaryStatistics{count=77, sum=2222.710000, min=2.500000, average=28.866364, m
ax=263.500000}
```

Limitar el numero de productos devueltos

```
products.stream().limit(50); // limitamos a 50 productos
```

Saltar hasta el elemento indicado y a partir de ahí devolver todos los elementos

```
Stream<Product> skip = products.stream().skip(5); //obtenemos los productos a partir del
6 (inclusive)
skip.forEach(System.out::println);

// Product{id=6, name='Grandma's Boysenberry Spread', supplier=3, category=2...}
// Product{id=7, name='Uncle Bob's Organic Dried Pears', supplier=3, category=7...}
// Product{id=8, name='Northwoods Cranberry Sauce', supplier=3, category=2...}
// ...
```

10. Resumen

Hemos visto como a partir de Java 8 podemos ejecutar sobre colecciones potentes operaciones de agregación como en SQL. Para mejor comprensión debes escribir estos ejemplos tu mismo y ver los resultados. Si lo deseas incluso puedes cargar el archivo csv a una base de datos relacional para comprobar los resultados.

Podemos decir que,

- `Stream` Representa un flujo de elementos y podemos aplicar métodos tales como
  - `filter` para filtrar los elementos. Éste método recibe un predicado como argumento.
  - `map` para devolver solo el atributo indicado. Es como el select de sql.
  - `sorted` para ordenar nuestros elementos. Recibe un `Comparator`.
  - `min`, `max`, `count` que permiten obtener el minimo, máximo y conteo de elementos respectivamente.
  - `collect` aquí es donde definiremos nuestros funciones de agregado, principalmente agrupados, particiones, etc.
- `Comparator` nos proporciona métodos útiles para realizar ordenamientos. Lo usaremos en conjunto con el método `sorted`
- `Collectors` Nos proporciona métodos útiles para agrupar, sumar, promediar, obtener estadísticas. Lo usaremos en conjunto con el método `collect`

En esta primera parte aprendimos varias operaciones, la más potente es el uso de `collect`.

En la segunda parte de este workshop aprenderemos sobre el uso de otras 2 operaciones más potentes como son `reduce` y `flatmap`.

No olvides dejar tus comentarios.

Codigo fuente disponible de este tutorial (https://github.com/windoctor7/codigo-tutoriales-blog/tree/master/java-streams)

Otros post que pueden interesarte

April 27, 2019

Introduccion Flutter (/Introduccion-Flutter.html)

October 30, 2017

Programación Reactiva con Spring WebFlux - Parte 2 (/webflux-kotlin.html)

October 22, 2017

Desarrollando en Kotlin y Spring 5 (/intro-kotlin-spring.html)

2 Comentarios **Ascari Romo** **Acceder**

Recomendar 7 Tweet Compartir **Ordenar por los mejores**

Únete a la conversación...

INICIAR SESIÓN CON O REGISTRARSE CON DISQUS ?

Nombre

Alejandro • hace 5 meses

Muy claro. Muchas gracias!

• Responder • Compartir ›

Victor Cruz Gomez • hace un año

Excelente...

• Responder • Compartir ›

Suscríbete **Añade Disqus a tu sitio webAñade Disqus Añadir**

© All rights reserved. Powered by Jekyll (http://jekyllrb.com/) and sustain (http://www.github.com/biomadeira/sustain) with ♥